Principe et Implémentation d'un Réseau de Neurones Artificiel

Mohamed Ahmed Mohamed Lemine

9 octobre 2025

Résumé

Les réseaux de neurones artificiels (ANN) constituent la base du *Deep Learning*. Ce document présente les principes fondamentaux d'un neurone artificiel, la modélisation mathématique d'un réseau multicouche (MLP), les principales fonctions de coût, ainsi que les méthodes d'optimisation comme la descente de gradient et ses variantes. L'algorithme de rétropropagation est ensuite détaillé avant de présenter une implémentation en Python et une application sur le dataset MNIST pour la reconnaissance de chiffres manuscrits.

Table des matières

1 Introduction **2**
- 1.1 Définition d'un neurone . . . 2
- 1.2 Les fonctions d'activation . . . 2
- 1.3 Généralisation : le Perceptron Multicouche (MLP) . . . 5

2 Fonction de coût **7**
- 2.1 Erreur quadratique moyenne (MSE) . . . 7

3 Algorithme de Backpropagation **9**
- 3.1 Définition . . . 9
- 3.2 Propriétés fondamentales . . . 10

4 Alog **16**

5 Implémentation en Python **17**

6 Application : MNIST **18**

1. Introduction

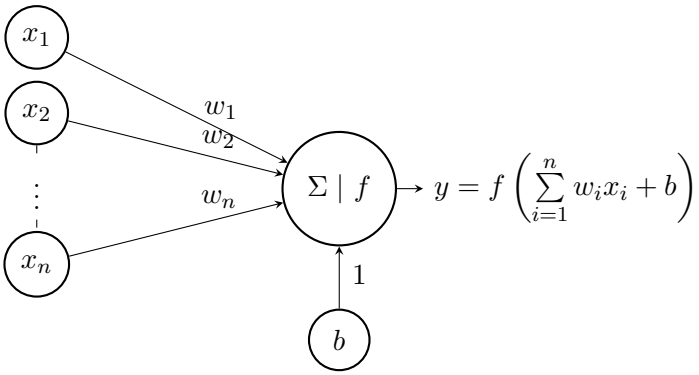
1.1. Définition d'un neurone

Un neurone artificiel est une unité de calcul inspirée du neurone biologique. Il prend des entrées $x_1, x_2, \ldots, x_n$, les pondère par des coefficients w_i, ajoute un biais b, et applique une fonction d'activation f :

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

L'objectif est d'apprendre les poids w_i et le biais b pour minimiser l'erreur entre la sortie prédite et la valeur réelle.

Voici une représentation schématique d'un neurone artificiel :



1.2. Les fonctions d'activation

Définition

Une **fonction d'activation** introduit de la *non-linéarité* dans un réseau de neurones. Sans cette non-linéarité, un réseau multicouche se comporterait comme une simple régression linéaire, incapable de modéliser des relations complexes.

Ainsi, chaque neurone applique une fonction f sur la combinaison linéaire de ses entrées :

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

Les principales fonctions d'activation utilisées dans la pratique sont : la sigmoïde, la tangente hyperbolique, ReLU et softmax.

a. Sigmoïde

La fonction sigmoïde transforme toute valeur réelle en une valeur comprise entre 0 et 1 :

$$f(x) = \frac{1}{1 + e^{-x}}$$

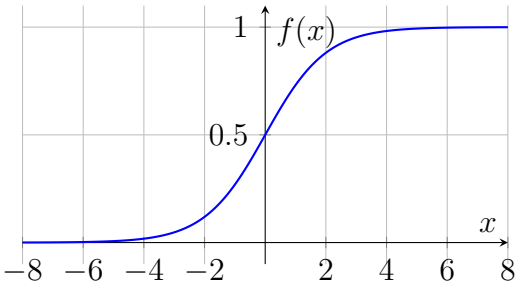
Dérivée :

$$f'(x) = \frac{e^{-x}}{(1 + e^{-x})^2} = f(x)(1 - f(x))$$

Propriétés :

- Bornée : $f(x) \in]0, 1[$
- Lisse et monotone croissante
- Bonne interprétation probabiliste
- *Inconvénient* : saturations pour $|x|$ grands $\Rightarrow$ gradients proches de 0 (vanishing gradient)

Sigmoïde



b. Tangente hyperbolique (Tanh)

La fonction Tanh est une version centrée de la sigmoïde :

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

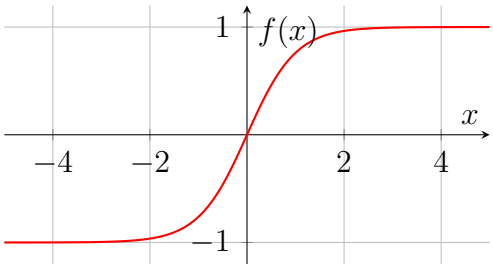
Dérivée :

$$f'(x) = 1 - f(x)^2$$

Propriétés :

- Bornée : $f(x) \in]-1, 1[$
- Zéro-centre la sortie (accélère l'apprentissage)
- *Inconvénient* : toujours sujette au problème de gradient qui disparaît pour $|x|$ grand

Tangente hyperbolique



c. ReLU (Rectified Linear Unit)

La fonction ReLU est définie comme :

$$f(x) = \max(0, x)$$

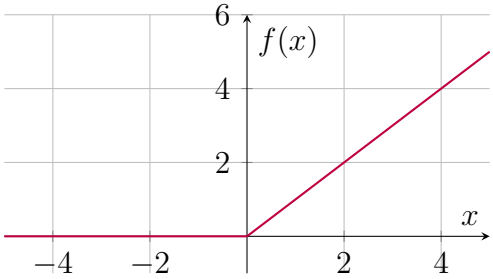
Dérivée :

$$f'(x) = \begin{cases} 1 & \text{si } x > 0, \\ 0 & \text{sinon.} \end{cases}$$

Propriétés :

- Simple et très efficace pour les réseaux profonds
- Pas de saturation pour les valeurs positives
- *Inconvénient* : les neurones peuvent "mourir" si $x < 0$ trop souvent (*dying ReLU*)

ReLU



Résumé :

Fonction	Forme	Plage	Dérivée principale
Sigmoïde	$\frac{1}{1+e^{-x}}$	(0,1)	$f(1-f)$
Tanh	$\tanh(x)$	(-1,1)	$1-f^2$
ReLU	$\max(0,x)$	[0,∞)	$1_{x>0}$

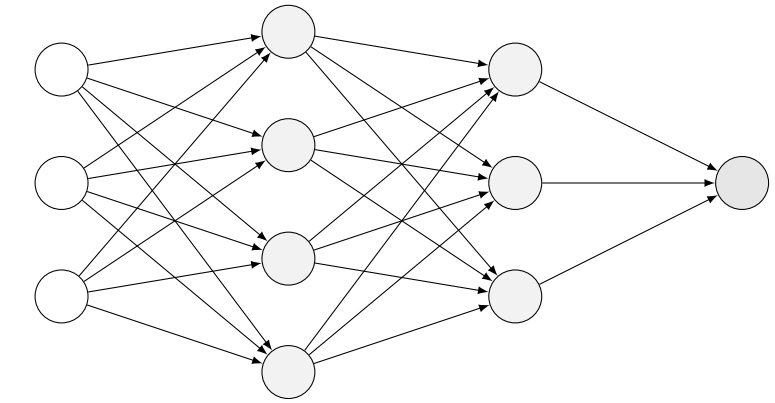
1.3. Généralisation : le Perceptron Multicouche (MLP)

Définition

Un **Perceptron Multicouche** (ou *Multi-Layer Perceptron*, MLP) est une extension naturelle du neurone artificiel à plusieurs couches. Il est constitué :

- d'une **couche d'entrée**, qui reçoit le vecteur des caractéristiques $x \in \mathbb{R}^{n_0}$,
- d'une ou plusieurs **couches cachées**, chacune composée de plusieurs neurones,
- d'une **couche de sortie**, qui produit la prédiction finale $\hat{y}$.

Chaque couche effectue une transformation **affine** suivie d'une **fonction d'activation non linéaire**. Ainsi, la sortie d'une couche devient l'entrée de la couche suivante.



Entrée x Couches cachées Sortie $\hat{y}$

Notations

On note :

- L : le nombre total de couches (hors entrée),
- n_l : le nombre de neurones dans la couche l,
- $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$: la matrice des poids reliant la couche $(l-1)$ à la couche l,
- $b^{(l)} \in \mathbb{R}^{n_l}$: le vecteur des biais de la couche l,
- $a^{(l)} \in \mathbb{R}^{n_l}$: le vecteur des activations (sorties) de la couche l.

Ainsi, pour la couche l :

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \qquad a^{(l)} = f^{(l)}(z^{(l)}),$$

où $f^{(l)}$ est la fonction d'activation choisie pour la couche l.

Formulation générale du MLP

La propagation avant (*forward propagation*) d'un réseau MLP s'écrit de manière récursive :

$$\begin{cases} a^{(0)} = x, & \text{(entrée)} \\ a^{(l)} = f^{(l)}(W^{(l)}a^{(l-1)} + b^{(l)}), & \forall l = 1, 2, \ldots, L-1 \\ \hat{y} = f^{(L)}(W^{(L)}a^{(L-1)} + b^{(L)}), & \text{(sortie finale)} \end{cases}$$

où :

- $f^{(l)}$ peut varier selon les couches
- $\hat{y}$ est la prédiction du réseau,
- le vecteur x est l'entrée du modèle.

On peut résumer la transformation complète du réseau comme une **composition de fonctions de couche**. Pour chaque couche l, on définit :

$$g^{(l)}(x) := f^{(l)}\big(W^{(l)}x + b^{(l)}\big)$$

où $g^{(l)} : \mathbb{R}^{n_{l-1}} \to \mathbb{R}^{n_l}$ représente la transformation complète réalisée par la couche l.

Ainsi, la sortie finale du réseau (*forward pass*) s'écrit comme une composition de ces transformations :

$$\hat{y} = g^{(L)} \circ g^{(L-1)} \circ \cdots \circ g^{(1)}(x)$$

Remarques

- Le MLP est un **approximateur universel** : avec suffisamment de neurones et de couches, il peut approximer n'importe quelle fonction continue.
- Le nombre de couches et de neurones influence la capacité du modèle et sa tendance au surapprentissage.
- Les fonctions d'activation assurent la non-linéarité nécessaire pour modéliser des relations complexes.

2. Fonction de coût

Introduction

L'objectif de l'apprentissage supervisé est d'ajuster les paramètres du modèle (poids et biais) afin de minimiser l'erreur entre les prédictions du réseau $\hat{y}$ et les vraies valeurs y. Pour quantifier cette erreur, on introduit une **fonction de coût** (ou *loss function*) notée $L(y, \hat{y})$.

Une bonne fonction de coût doit :

- être différentiable (pour permettre l'optimisation par descente de gradient),
- croître lorsque les prédictions s'éloignent de la vérité,
- refléter correctement la nature du problème (classification, régression, etc.).

Les fonctions de coût les plus utilisées sont :

- **MSE (Mean Squared Error)** pour les tâches de régression ;
- **Cross-Entropy** pour les tâches de classification.

2.1. Erreur quadratique moyenne (MSE)

Définition

L'erreur quadratique moyenne (MSE) mesure la moyenne des carrés des écarts entre les valeurs réelles et prédites :

$$L_{MSE}(y, \hat{y}) = \frac{1}{N} \sum_{i=1}^{N} (y_i - \hat{y}_i)^2$$

où :

- y_i : la valeur réelle de la $i^{\text{ème}}$ observation,
- $\hat{y}_i$: la valeur prédite correspondante,
- N : le nombre total d'échantillons.

La MSE pénalise fortement les grandes erreurs, car celles-ci sont élevées au carré.

Définition dans le contexte des réseaux de neurones

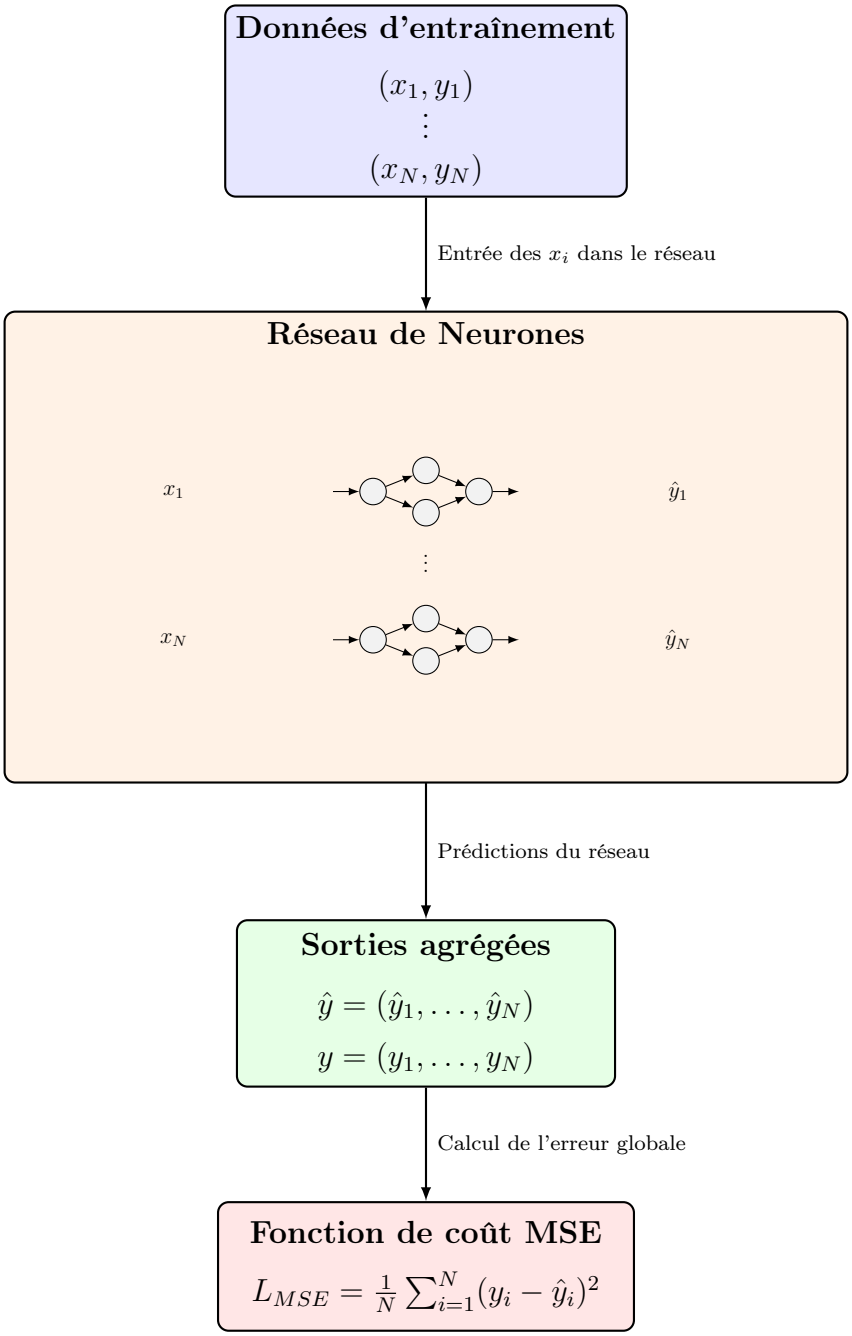
Dans un réseau de neurones, la MSE évalue la différence entre les sorties du réseau $\hat{y}^{(i)} = \mathcal{F}(x^{(i)}; \Theta)$ et les vraies cibles $y^{(i)}$ pour l'ensemble des échantillons du jeu d'entraînement :

$$L_{MSE}(\Theta) = \frac{1}{N} \sum_{i=1}^{N} \left\| y^{(i)} - \mathcal{F}(x^{(i)}; \Theta) \right\|_2^2$$

où :

- $\mathcal{F}(x^{(i)}; \Theta)$ désigne la sortie du réseau pour l'entrée $x^{(i)}$ avec paramètres $\Theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$,
- $y^{(i)}$ est le vecteur de sortie attendu (valeur réelle),
- $\| \cdot \|_2$ représente la norme euclidienne.

Cette formulation montre que la fonction de coût dépend implicitement des poids et biais du réseau à travers $\mathcal{F}(x^{(i)}; \Theta)$. Minimiser $L_{MSE}(\Theta)$ revient donc à ajuster les paramètres Θ afin que le réseau reproduise au mieux les valeurs cibles.



3. Algorithme de Backpropagation

3.1. Définition

L'**algorithme de rétropropagation du gradient** (ou *Backpropagation*) est une méthode permettant de calculer efficacement les gradients de la fonction de coût par rapport aux paramètres du réseau de neurones (poids et biais).

Il s'agit du cœur de l'apprentissage des réseaux de neurones, car il permet la mise à jour des paramètres via une méthode d'optimisation (comme la descente de gradient).

- Il repose sur l'application répétée de la **règle de la chaîne** pour propager l'erreur depuis la couche de sortie jusqu'aux couches d'entrée.
- L'objectif est de minimiser la fonction de perte $L(\Theta)$ en ajustant les paramètres $\Theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$.

Notations générales

Soit un réseau de neurones comportant L couches (hors couche d'entrée). Pour chaque couche $l \in \{1, 2, \ldots, L\}$, on définit les éléments suivants :

- n_l : nombre de neurones dans la couche l.
- $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$: matrice des poids reliant la couche $(l-1)$ à la couche l :

$$W^{(l)} = \begin{bmatrix} w_{1,1}^{(l)} & w_{1,2}^{(l)} & \cdots & w_{1,n_{l-1}}^{(l)} \\ w_{2,1}^{(l)} & w_{2,2}^{(l)} & \cdots & w_{2,n_{l-1}}^{(l)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_l,1}^{(l)} & w_{n_l,2}^{(l)} & \cdots & w_{n_l,n_{l-1}}^{(l)} \end{bmatrix}$$

  où $w_{i,j}^{(l)}$ représente le poids reliant la sortie du neurone j de la couche $(l-1)$ à l'entrée du neurone i de la couche l.

- $b^{(l)} \in \mathbb{R}^{n_l}$: vecteur des biais de la couche l :

$$b^{(l)} = \begin{bmatrix} b_1^{(l)} \\ b_2^{(l)} \\ \vdots \\ b_{n_l}^{(l)} \end{bmatrix}$$

- $f^{(l)} : \mathbb{R}^{n_l} \to \mathbb{R}^{n_l}$: fonction d'activation appliquée élément par élément :

$$f^{(l)}(z) = \begin{bmatrix} f^{(l)}(z_1) \\ f^{(l)}(z_2) \\ \vdots \\ f^{(l)}(z_{n_l}) \end{bmatrix}$$

- $a^{(l)} \in \mathbb{R}^{n_l}$: vecteur des activations (sorties) de la couche l.
- $z^{(l)} \in \mathbb{R}^{n_l}$: vecteur des combinaisons linéaires avant activation :

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)} \quad \text{et} \quad a^{(l)} = f^{(l)}(z^{(l)}).$$

Objectif

Soit $L(\Theta)$ la **fonction de perte globale** du réseau, qui dépend de tous les paramètres $\Theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L}$:

$$L(\Theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(y^{(i)}, \hat{y}^{(i)}(\Theta)\big)$$

où ℓ est la perte pour un échantillon donné (par exemple MSE ou Cross-Entropy), et $\hat{y}^{(i)}(\Theta)$ est la sortie du réseau pour l'entrée $x^{(i)}$.

L'objectif de la rétropropagation est de fournir une **méthode récursive** pour calculer, pour chaque couche $l \in \{1, \ldots, L\}$, les gradients

$$\frac{\partial L}{\partial W^{(l)}} \quad \text{et} \quad \frac{\partial L}{\partial b^{(l)}},$$

afin de pouvoir **mettre à jour** les paramètres du réseau et **minimiser** la fonction de perte L.

Gradients par couche. Pour la couche l de taille n_l (avec n_{l-1} entrées), on définit :

$$\frac{\partial L}{\partial W^{(l)}} = \begin{bmatrix} \frac{\partial L}{\partial w_{1,1}^{(l)}} & \cdots & \frac{\partial L}{\partial w_{1,n_{l-1}}^{(l)}} \\ \vdots & \ddots & \vdots \\ \frac{\partial L}{\partial w_{n_l,1}^{(l)}} & \cdots & \frac{\partial L}{\partial w_{n_l,n_{l-1}}^{(l)}} \end{bmatrix} \in \mathbb{R}^{n_l \times n_{l-1}}, \qquad \frac{\partial L}{\partial b^{(l)}} = \begin{bmatrix} \frac{\partial L}{\partial b_1^{(l)}} \\ \vdots \\ \frac{\partial L}{\partial b_{n_l}^{(l)}} \end{bmatrix} \in \mathbb{R}^{n_l}.$$

Autrement dit, $\big(\frac{\partial L}{\partial W^{(l)}}\big)_{i,j} = \frac{\partial L}{\partial w_{i,j}^{(l)}}$ et $\big(\frac{\partial L}{\partial b^{(l)}}\big)_i = \frac{\partial L}{\partial b_i^{(l)}}$.

3.2. Propriétés fondamentales

Motivation. Le but du calcul de rétropropagation est d'obtenir les dérivées de la fonction de coût globale

$$L(\Theta) = \frac{1}{N} \sum_{i=1}^{N} \ell\big(y^{(i)}, \hat{y}^{(i)}(\Theta)\big)$$

par rapport aux paramètres du réseau. Ainsi, pour chaque couche $l \in \{1, \dots, L\}$:

$$\frac{\partial L}{\partial W^{(l)}} = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \ell\big(y^{(i)}, \hat{y}^{(i)}\big)}{\partial W^{(l)}}, \qquad \frac{\partial L}{\partial b^{(l)}} = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \ell\big(y^{(i)}, \hat{y}^{(i)}\big)}{\partial b^{(l)}}.$$

Pour alléger les notations, on considère dans la suite un seul échantillon d'apprentissage (x, y), avec $x = a^{(0)}$ et $\ell(y, \hat{y})$ la perte correspondante. Les dérivées obtenues pour un échantillon se généralisent ensuite par moyenne sur le lot (mini-batch ou dataset complet).

Définition de $\delta^{(l)}$. Pour chaque couche $l \in \{1, \dots, L\}$, on définit :

$$\boxed{\delta^{(l)} := \frac{\partial \ell}{\partial z^{(l)}} \in \mathbb{R}^{n_l}}$$

Le vecteur $\delta^{(l)}$ est appelé *erreur de la couche l* : il mesure la sensibilité de la perte locale ℓ aux pré-activations de la couche l.

Propriétés :

Pour chaque couche l, on a les relations matricielles :

$$\boxed{\begin{aligned} \frac{\partial \ell}{\partial W^{(l)}} &= \delta^{(l)} \left(a^{(l-1)}\right)^{\top}, \\ \frac{\partial \ell}{\partial b^{(l)}} &= \delta^{(l)}. \end{aligned}}$$

RQ : Les dimensions sont cohérentes :

$$\delta^{(l)} \in \mathbb{R}^{n_l}, \quad a^{(l-1)} \in \mathbb{R}^{n_{l-1}} \;\Rightarrow\; \delta^{(l)} \big(a^{(l-1)}\big)^{\top} \in \mathbb{R}^{n_l \times n_{l-1}}.$$

Preuve :

calcul de $\dfrac{\partial \ell}{\partial W^{(l)}}$?

D'après la règle de la chaîne, on peut écrire :

$$\frac{\partial \ell}{\partial W^{(l)}} = \frac{\partial \ell}{\partial z^{(l)}} \frac{\partial z^{(l)}}{\partial W^{(l)}}.$$

où la quantité $\dfrac{\partial z^{(l)}}{\partial W^{(l)}}$ est un **tenseur d'ordre 3**, puisque $z^{(l)} \in \mathbb{R}^{n_l}$ et $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$:

$$\frac{\partial z^{(l)}}{\partial W^{(l)}} = \left(\frac{\partial z_k^{(l)}}{\partial w_{i,j}^{(l)}} \right)_{k,i,j} \in \mathbb{R}^{n_l \times n_l \times n_{l-1}}.$$

On a que la contraction entre un vecteur et un tenseur, si $T \in \mathbb{R}^{n_1 \times n_2 \times n_3}$ et $v \in \mathbb{R}^{n_1}$, est définit par :

$$v \cdot T = U \in \mathbb{R}^{n_2 \times n_3} \quad \text{avec} \quad U_{i,j} = \sum_{k=1}^{n_1} v_k T_{k,i,j}.$$

En appliquant cette règle ici :

$$\frac{\partial \ell}{\partial W^{(l)}} = \sum_{k=1}^{n_l} \frac{\partial \ell}{\partial z_k^{(l)}} \frac{\partial z_k^{(l)}}{\partial W^{(l)}}.$$

On calcule alors explicitement les dérivées partielles de $z^{(l)}$:

$$z_k^{(l)} = \sum_{r=1}^{n_{l-1}} w_{k,r}^{(l)} a_r^{(l-1)} + b_k^{(l)},$$

d'où :

$$\frac{\partial z_k^{(l)}}{\partial w_{i,j}^{(l)}} = \mathbf{1}_{k=i}\, a_j^{(l-1)}.$$

En remplaçant dans l'expression précédente :

$$\frac{\partial \ell}{\partial w_{i,j}^{(l)}} = \sum_{k=1}^{n_l} \frac{\partial \ell}{\partial z_k^{(l)}} \frac{\partial z_k^{(l)}}{\partial w_{i,j}^{(l)}} = \sum_{k=1}^{n_l} \delta_k^{(l)}\, \mathbf{1}_{k=i}\, a_j^{(l-1)} = \delta_i^{(l)}\, a_j^{(l-1)}.$$

Sous forme matricielle, les éléments de la matrice gradient $\frac{\partial \ell}{\partial W^{(l)}}$ s'écrivent :

$$\left(\frac{\partial \ell}{\partial W^{(l)}}\right)_{i,j} = \delta_i^{(l)}\, a_j^{(l-1)}.$$

Ce qui correspond exactement au **produit extérieur** entre $\delta^{(l)}$ et $a^{(l-1)}$:

$$\boxed{\frac{\partial \ell}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^\top.}$$

Calcul de $\dfrac{\partial \ell}{\partial b^{(l)}}$?

même raisonnement que la partie précédente donc on a :

$$\frac{\partial z_k^{(l)}}{\partial b_i^{(l)}} = \mathbf{1}_{k=i}.$$

Donc :

$$\frac{\partial \ell}{\partial b_i^{(l)}} = \sum_{k=1}^{n_l} \frac{\partial \ell}{\partial z_k^{(l)}} \frac{\partial z_k^{(l)}}{\partial b_i^{(l)}} = \sum_{k=1}^{n_l} \delta_k^{(l)} \mathbf{1}_{k=i} = \delta_i^{(l)}.$$

Sous forme vectorielle : $\boxed{\frac{\partial \ell}{\partial b^{(l)}} = \delta^{(l)}}$.

Algorithme de calcul de $\delta^{(l)}$

Des résultats précédents, pour chaque couche l :

$$\frac{\partial \ell}{\partial W^{(l)}} = \delta^{(l)}(a^{(l-1)})^\top, \qquad \frac{\partial \ell}{\partial b^{(l)}} = \delta^{(l)}.$$

Ainsi, le problème du calcul des gradients se ramène à calculer $\delta^{(l)}$ pour toutes les couches.

Propriétés : calcul de $\delta^{(l)}$

alog pour le calcul $\left(\delta^{(l)}\right)_{1\leq l\leq L}$:

$$\begin{cases} \delta^{(L)} = \frac{\partial \ell}{\partial \hat{y}} \odot f'^{(L)}\left(z^{(L)}\right), \quad \text{où } \hat{y} = a^{(L)} \\ \delta^{(l)} = \left((W^{(l+1)})^\top \delta^{(l+1)}\right) \odot f'^{(l)}\left(z^{(l)}\right) \quad \forall l \in 1, \ldots, L-1 \end{cases}$$

Avec $\odot$ est le produit de Hadamard definit par : $u \odot v = \left(u_i v_i\right)_{i\leq n} \forall u, v \in \mathbb{R}^n$

Preuve :

- Calcul de $\delta^{(L)}$:

En appliquant la règle de la chaîne, pour tout $i \in \{1, \ldots, n_L\}$:

$$\delta_i^{(L)} = \frac{\partial \ell}{\partial z_i^{(L)}} = \frac{\partial \ell}{\partial a_i^{(L)}} \cdot \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}}.$$

Or, comme $a^{(L)} = f^{(L)}(z^{(L)})$, on a pour tout $i \in \{1, \ldots, n_L\}$:

$$a_i^{(L)} = f^{(L)}(z_i^{(L)}) \quad \Rightarrow \quad \frac{\partial a_i^{(L)}}{\partial z_i^{(L)}} = f'^{(L)}(z_i^{(L)}).$$

En substituant dans l'équation précédente, on obtient :

$$\delta_i^{(L)} = \frac{\partial \ell}{\partial a_i^{(L)}} f'^{(L)}(z_i^{(L)}).$$

Ainsi, sous forme vectorielle, la relation s'écrit :

$$\boxed{\delta^{(L)} = \frac{\partial \ell}{\partial a^{(L)}} \odot f'^{(L)}(z^{(L)}),}$$

- Calcul de $\delta^{(l)}$:

Pour $l \in \{1, \ldots, L-1\}$, on veut exprimer $\delta^{(l)} = \frac{\partial \ell}{\partial z^{(l)}}$ en fonction de $\delta^{(l+1)}$.

On a $a^{(l)} = f^{(l)}(z^{(l)})$ et $z^{(l+1)} = W^{(l+1)}a^{(l)} + b^{(l+1)}$. Appliquons la règle de la chaîne en "remontant" une couche :

$$\frac{\partial \ell}{\partial z^{(l)}} = \Big(\frac{\partial a^{(l)}}{\partial z^{(l)}}\Big)^\top \Big(\frac{\partial z^{(l+1)}}{\partial a^{(l)}}\Big)^\top \frac{\partial \ell}{\partial z^{(l+1)}}.$$

Or

$$\frac{\partial a^{(l)}}{\partial z^{(l)}} = \mathrm{Diag}\big(f'^{(l)}(z^{(l)})\big) \quad \text{et} \quad \frac{\partial z^{(l+1)}}{\partial a^{(l)}} = W^{(l+1)}.$$

En notant $\delta^{(l+1)} = \frac{\partial \ell}{\partial z^{(l+1)}}$, on obtient

$$\delta^{(l)} = \mathrm{Diag}\big(f'^{(l)}(z^{(l)})\big)\ (W^{(l+1)})^\top \delta^{(l+1)}.$$

Comme la multiplication par une matrice diagonale revient à une multiplication élément par élément, ceci s'écrit

$$\boxed{\delta^{(l)} = \big((W^{(l+1)})^\top \delta^{(l+1)}\big)\ \odot\ f'^{(l)}(z^{(l)}).}$$

Algorithm 1: Rétropropagation (un seul exemple (x, y), avec $a^{(0)} = x$)

Entrée: $\{W^{(l)}, b^{(l)}\}_{l=1}^{L}$, l'exemple (x, y), activations $\{f^{(l)}\}$

Sortie : Gradients $\left\{\frac{\partial \ell}{\partial W^{(l)}}, \frac{\partial \ell}{\partial b^{(l)}}\right\}_{l=1}^{L}$

Définir la perte (MSE) : $\ell(y, \hat{y}) = \frac{1}{2}\|y - \hat{y}\|_2^2$ // $\Rightarrow$ $\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y$

Propagation avant : **for** $l = 1$ **to** L **do**

  $z^{(l)} \leftarrow W^{(l)} a^{(l-1)} + b^{(l)}$

  $a^{(l)} \leftarrow f^{(l)}\big(z^{(l)}\big)$

end

$\hat{y} \leftarrow a^{(L)}$

Erreur en sortie : // $\hat{y} = a^{(L)}$

$\delta^{(L)} \leftarrow \dfrac{\partial \ell}{\partial \hat{y}} \odot f'^{(L)}\big(z^{(L)}\big)$

(pour MSE : $\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y$*)*

Propagation en arrière : **for** $l = L - 1$ 1 **do**

  $\delta^{(l)} \leftarrow \big((W^{(l+1)})^\top \delta^{(l+1)}\big) \odot f'^{(l)}\big(z^{(l)}\big)$

end

Gradients : **for** $l = 1$ **to** L **do**

  $\dfrac{\partial \ell}{\partial W^{(l)}} \leftarrow \delta^{(l)} \, (a^{(l-1)})^\top$

  $\dfrac{\partial \ell}{\partial b^{(l)}} \leftarrow \delta^{(l)}$

end

Mise à jour SGD : **for** $l = 1$ **to** L **do**

  $W^{(l)} \leftarrow W^{(l)} - \eta \, \frac{\partial \ell}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \, \frac{\partial \ell}{\partial b^{(l)}}$

end

RQ : Cas d'un mini-lot (batch) de taille m Pour un mini-lot $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$:

$$\frac{\partial L}{\partial W^{(l)}} = \frac{1}{m} \sum_{i=1}^{m} \frac{\partial \ell^{(i)}}{\partial W^{(l)}} = \frac{1}{m} \sum_{i=1}^{m} \delta_{(i)}^{(l)} (a_{(i)}^{(l-1)})^\top, \qquad \frac{\partial L}{\partial b^{(l)}} = \frac{1}{m} \sum_{i=1}^{m} \delta_{(i)}^{(l)}.$$

Ces gradients moyens servent ensuite à la mise à jour des paramètres par descente de gradient :

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial L}{\partial W^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial L}{\partial b^{(l)}}.$$

4. Alog

Les différentes approches d'entraînement des réseaux de neurones

L'entraînement d'un réseau de neurones consiste à minimiser la perte moyenne sur un ensemble de données

$$\mathcal{L}(W) = \frac{1}{N} \sum_{i=1}^{N} \ell_i(W)$$

où $\ell_i(W) = \mathcal{L}(y^{(i)}, \hat{y}^{(i)}(W, x^{(i)}))$ représente la perte pour l'exemple $(x^{(i)}, y^{(i)})$.

Selon la manière dont on calcule et met à jour les gradients, on distingue trois approches principales :

1. Batch Gradient Descent (Descente de gradient par lot complet)

— On effectue un **forward pass** sur *toutes les données* :

$$x^{(1)}, \ldots, x^{(N)} \;\Rightarrow\; \hat{y}^{(1)}, \ldots, \hat{y}^{(N)}$$

— On calcule la perte moyenne :

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \ell_i$$

— On effectue **un seul backward pass** :

$$\frac{\partial \mathcal{L}}{\partial W^{(l)}} = \frac{1}{N} \sum_{i=1}^{N} \frac{\partial \ell_i}{\partial W^{(l)}}$$

— Mise à jour :

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$

Avantages : gradient exact, convergence stable. **Inconvénients :** très coûteux en mémoire et en calcul pour les grands ensembles de données.

2. Stochastic Gradient Descent (Descente de gradient stochastique)

— On sélectionne un seul exemple $(x^{(i)}, y^{(i)})$.

— On effectue un **forward pass** :

$$x^{(i)} \;\Rightarrow\; \hat{y}^{(i)}$$

— On calcule la perte ℓ_i et on effectue un **backward pass** :

$$\nabla_W \ell_i$$

— Mise à jour immédiate :

$$W \leftarrow W - \eta \nabla_W \ell_i$$

Avantages : faible coût mémoire, mise à jour rapide. **Inconvénients :** gradient très bruité, trajectoire instable.

3. Mini-Batch Gradient Descent (Descente de gradient par mini-lots)

— On divise le dataset en lots (*batches*) de taille m.

— Pour chaque lot :

$$A^{(l-1)} = [a^{(l-1)}(1), \ldots, a^{(l-1)}(m)] \in \mathbb{R}^{n_{l-1} \times m}$$
$$\Delta^{(l)} = [\delta^{(l)}(1), \ldots, \delta^{(l)}(m)] \in \mathbb{R}^{n_l \times m}$$

— On effectue un seul backward pass pour le lot :

$$\frac{\partial \mathcal{L}_{\text{batch}}}{\partial W^{(l)}} = \frac{1}{m} \Delta^{(l)} (A^{(l-1)})^T$$

— Mise à jour :

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{L}_{\text{batch}}}{\partial W^{(l)}}$$

Avantages : compromis entre précision et rapidité, efficace sur GPU, convergence stable. **Inconvénients :** nécessite le choix d'une taille de batch adaptée.

Résumé comparatif

Méthode	Forward sur	Backward sur	Mise à jour après
Batch	Tous les exemples	Tous les exemples	1 itération
Stochastique	1 exemple	1 exemple	Chaque exemple
Mini-batch	m exemples	m exemples	Chaque batch

En pratique, la **descente de gradient par mini-batch** est la plus utilisée, car elle exploite les calculs vectorisés et assure une convergence plus stable tout en restant efficace en temps de calcul.

5. Implémentation en Python

Voici un pseudocode minimal pour un MLP from scratch :

```
class Layer:
    def __init__(self, in_dim, out_dim, activation):
        self.W = np.random.randn(out_dim, in_dim) * 0.01
        self.b = np.zeros((out_dim, 1))
        self.activation = activation

    def forward(self, X):
        self.Z = np.dot(self.W, X) + self.b
        self.A = self.activation(self.Z)
        return self.A

    def backward(self, dA, lr):
        dZ = dA * self.activation.derivative(self.Z)
        dW = np.dot(dZ, self.A_prev.T)
        db = np.sum(dZ, axis=1, keepdims=True)
        self.W -= lr * dW
        self.b -= lr * db
```

6. Application : MNIST

Le dataset MNIST contient 70 000 images de chiffres manuscrits (28x28). Un réseau simple :

— Couche 1 : $784 \to 128$ (ReLU)

— Couche 2 : $128 \to 64$ (ReLU)

— Couche 3 : $64 \to 10$ (Softmax)

permet d'obtenir environ 97% de précision.

